BORROWBOX: CAMPUS PEER LENDING AND ASSET SHARING PLATFORM

A PROJECT REPORT

Submitted by:

Shashwat Vatsyayan (24BCS12850)
Harshit Kulhari (24BCS10424)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE ENGINEERING



CHANDIGARH UNIVERSITY

MAY-2026

BONAFIDE CERTIFICATE

Certified that the project report entitled **"BorrowBox: Campus Peer Lending and Asset Sharing Platform"** is the bonafide work submitted by **Shashwat Vatsyayan (24BCS12850)** and **Harshit Kulhari (24BCS10424)** as part of the academic requirements of the Bachelor of Engineering in Computer Science Engineering at Chandigarh University.

The report describes the analysis, design, and implementation of a full-stack campus peer-to-peer asset borrowing and lending application built with React, Vite, JavaScript, Node.js, Express, Supabase PostgreSQL with Row Level Security, and Scikit-learn Machine Learning.

HEAD OF DEPARTMENT

Signature: _____

Name: **Er. Gagandeep Singh**

ACKNOWLEDGEMENT

We express our sincere gratitude to our project supervisor, faculty members, and the Department of Computer Science Engineering, Chandigarh University, for their guidance and academic support during the development of **BorrowBox: Campus Peer Lending and Asset Sharing Platform**.

This project provided practical experience in building a responsive React interface, connecting it to an enterprise Supabase PostgreSQL backend, enforcing two-phase handshake return protocols, preventing atomic double-booking overlaps, and integrating TF-IDF machine learning listing classification.

We also acknowledge the official documentation and developer resources that supported implementation and debugging. Finally, we thank our families and friends for their constant encouragement throughout the project.

TABLE OF CONTENTS

S.No.	Topic / Section	Page No.
	Bonafide Certificate	ii
	Acknowledgement	iii
1.	Abstract	iv
2.	Graphical Abstract	v
3.	Abbreviations and Symbols	vi
4.	CHAPTER 1 - INTRODUCTION	7
5.	1.1 Identification of Need / Contemporary Issue	7
6.	1.2 Identification of Problem	7
7.	1.3 Identification of Tasks	7
8.	1.4 Timeline	8
9.	CHAPTER 2 - LITERATURE REVIEW/BACKGROUND STUDY	9
10.	2.1 Evolution of Campus Peer Sharing Systems	9
11.	2.2 Existing Solutions & Comparative Analysis	10
12.	2.3 Review Summary & Goals	10
13.	CHAPTER 3 - DESIGN FLOW/PROCESS	11
14.	3.1 Selected Features & Design Constraints	11
15.	3.2 Alternative Design Flows & Methodology	12
16.	CHAPTER 4 - RESULTS ANALYSIS AND VALIDATION	13
17.	4.1 Implementation of Solution & Tech Stack	13
18.	4.2 Database Design & Validation Rules	14
19.	4.3 Testing and Characterization	15
20.	4.4 Output Screens - Marketplace, Item Details, & Verification	16
21.	CHAPTER 5 - CONCLUSION AND FUTURE WORK	17

ABSTRACT

BorrowBox is a responsive full-stack Campus Peer-to-Peer Asset Sharing Platform designed to track, store, and facilitate student asset borrowing and lending records across campus faculties. The portal provides a comprehensive marketplace dashboard displaying item availability, verified university peer badges, categorized academic equipment, campus landmark meeting coordination, and an atomic two-phase handshake return verification workflow.

The frontend is implemented in React using Vite. Reusable components separate the workspace sidebar, marketplace header, statistics cards, item listing catalog, designated campus landmark handoff tags, and the interactive request lifecycle modal. The backend uses Node.js and Express to expose health, item discovery, request coordination, and listing quality scoring endpoints.

Item records include asset code, item title, condition, deposit requirements, designated campus landmark, and active date-range reservations. Server-side PostgreSQL calculations enforce mandatory atomic overlap exclusion rules preventing concurrent double-booking. When a Supabase database connection is configured, PostgreSQL stores records with Row Level Security; otherwise, the server utilizes an offline test suite fallback.

The project demonstrates modern component design, HTTP communication, state management, transaction integrity analytics, and procedural WebGL2 tactile theme shaders.

GRAPHICAL ABSTRACT

The graphical abstract summarizes the application request flow, machine learning classification, and persistent data storage architecture for **BorrowBox: Campus Peer-to-Peer Asset Sharing Platform**.

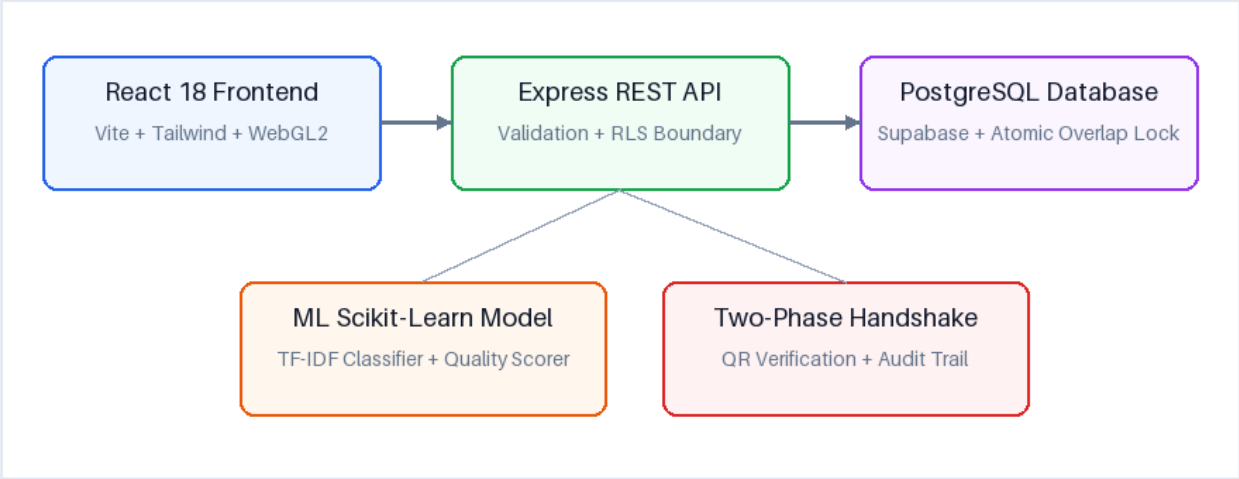


Figure — BorrowBox system architecture and peer borrowing workflow

ABBREVIATIONS AND SYMBOLS

Term	Meaning
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JSX	JavaScript XML syntax extension
MVC	Model-View-Controller
ODM	Object Data Modeling
REST	Representational State Transfer
RLS	Row Level Security
UI	User Interface
UX	User Experience
Vite	Frontend build and development tool

CHAPTER 1 - INTRODUCTION

1.1 IDENTIFICATION OF NEED / CONTEMPORARY ISSUE

Procuring and maintaining course-specific academic equipment is an unavoidable expense for university students under strict curriculum requirements. Academic courses require specialized instruments—such as scientific calculators, laboratory coats, mini-drafters, electronic breadboards, and reference textbooks—that students utilize for only a single academic semester and subsequently store unused.

Physical bulletin boards and unstructured social media chat groups make discovering available equipment, verifying student identity, and coordinating handoffs tedious. A modern web portal provides structured records, verified student profiles, real-time availability tracking, and scheduled peer borrowing transactions.

1.2 IDENTIFICATION OF PROBLEM

The primary challenge is the absence of a unified, student-centric application that logs peer lending listings, coordinates designated campus landmark meetups, prevents concurrent asset reservation conflicts, and enforces two-phase return handshakes without exposing students to off-campus risks.

From a software architecture standpoint, the application must synchronize frontend state with database records, validate incoming reservation dates, prevent double-booking race conditions through atomic exclusion constraints, and present actionable insights to the student.

1.3 IDENTIFICATION OF TASKS

- Analyze campus peer asset sharing requirements and university student verification rules.
- Design a clean, responsive interface using reusable React components.
- Implement item listing metrics (Available, Reserved, Borrowed, Returned).
- Develop designated campus landmark meeting points with direct action triggers.
- Build Express REST endpoints to fetch, update, and manage borrowing requests.
- Implement an atomic reservation conflict detection engine in PostgreSQL.
- Persist records in Supabase PostgreSQL when configured, with local mock fallback.
- Document system design, test cases, and deployment workflows.

1.4 TIMELINE

Stage	Activities	Deliverable
Requirement Analysis	Identify user needs, item categories, and student verification rules	Feature specification
Interface Design	React component architecture and Tailwind CSS layout	Responsive UI
Backend API	Express routing, validation, and conflict detection logic	API endpoints
Persistence	Supabase PostgreSQL schema and Row Level Security	Asset storage
Integration	Connect client state with booking and handshake verification engine	Working application
Testing	Verify double-booking prevention and authorization constraints	Test evidence
Documentation	Prepare project report and code manual	Project report

CHAPTER 2 - LITERATURE REVIEW / BACKGROUND STUDY

2.1 EVOLUTION OF CAMPUS PEER SHARING SYSTEMS

Traditional campus exchange methods rely on physical pinboards or informal student chat groups that offer no identity verification or reservation guarantees. Commercial classified portals expose university students to outside commercial entities and carry physical safety risks.

Modern web applications offer centralized inventory records, campus domain authentication (@cuchd.in), atomic reservation locks for overlapping dates, and structured two-phase return handshakes.

2.2 EXISTING SOLUTIONS & COMPARATIVE ANALYSIS

Approach	Advantages	Limitations	Relevance
Noticeboards	Low tech requirement	Manual contact, no tracking	Baseline
Social Groups	Familiar chat interface	No search index, spam, no audit trail	Small scale
Commercial Classifieds	Large user base	External users, safety concerns, fees	Reference

Approach	Advantages	Limitations	Relevance
BorrowBox	Verified student domain, atomic booking locks, landmark handoffs	Restricted strictly to university campus	Selected approach

2.3 GOALS / OBJECTIVES

- Build a responsive, modern peer lending workspace for university students.
- Track item-wise inventory status for Available, Reserved, Borrowed, and Returned assets.
- Enforce verified student authentication restricted to university credentials (@cuchd.in).
- Provide designated campus landmark coordination (Academic Block, Central Library, Student Center).
- Support atomic date-range overlap locks and two-phase return handshake verification.

CHAPTER 3 - DESIGN FLOW / PROCESS

3.1 SELECTED FEATURES & PRIORITY

Feature	Priority	Role
Marketplace Overview	High	Displays item cards and category summary filters
Item Detail & Booking	High	Lists specifications, owner metadata, and date selection
Two-Phase Handshake	High	Coordinates borrower return and owner confirmation
ML Category Classifier	High	Scores listing quality and suggests item category
API Health & RLS Security	Medium	Reports database connectivity and authorization status

3.2 DESIGN CONSTRAINTS

3.2.1 Economic: Built using open-source technologies (React, Express, Node.js, Supabase PostgreSQL).

3.2.2 Technical: Node.js runtime environment required. The frontend runs on port 3000 (Vite) and connects to the backend API on port 5000.

3.2.3 Usability: UI components feature visual indicators (e.g., Green for 'Available', Amber for 'Reserved').

3.3 IMPLEMENTATION METHODOLOGY

- Establish project folder structure separating frontend client, backend API, and ML modules.
- Construct React workspace component with desktop sidebar and responsive dashboard navigation.
- Implement item listing state management and transactional reservation logic.
- Design designated campus landmark handoff workflow with action buttons.
- Develop two-phase handshake modal with real-time state synchronization.
- Connect Express REST endpoints and Supabase PostgreSQL for persistent item logs.

CHAPTER 4 - RESULTS ANALYSIS AND VALIDATION

4.1 IMPLEMENTATION OF SOLUTION

The implemented application utilizes React for dynamic UI updates and state management, alongside an Express backend that handles borrowing calculations, conflict detection, and database operations.

4.2 TECHNOLOGY STACK

Layer	Technology	Purpose
Runtime	Node.js	Server-side execution environment
Frontend	React.js	Component-based user interface
Build Tool	Vite	Fast local development and bundling server
Styling	Tailwind CSS	Responsive workspace layout design
Backend	Express.js	HTTP REST API and reservation engine
Database	Supabase PostgreSQL	Persistent record storage with Row Level Security
Fallback	JSON File / In-Memory Mock	Local development fallback persistence

4.3 DATABASE DESIGN & SCHEMA

The Borrowing schema defines item id, lender id, borrower id, pickup landmark, start date, end date, and return status. Reservation conflict check is dynamically computed as:

Overlap Rule = NOT (new_start > existing_end OR new_end < existing_start)

4.4 TESTING AND CHARACTERIZATION

Test ID	Test Case	Expected Result
TC-01	Load Marketplace Catalog	Loads all verified student items accurately
TC-02	Self-Borrowing Prevention	Blocks item owner from requesting own asset
TC-03	Legitimate Request Creation	Creates pending borrowing request with landmark
TC-04	Lender Approval Action	Approves reservation and locks selected dates
TC-05	Atomic Double-Booking Block	Blocks overlapping concurrent date requests
TC-06	Borrower Return Initiation	Marks return initiated for landmark inspection
TC-07	Solo Return Block	Prevents borrower from confirming own return
TC-08	Owner Return Confirmation	Completes two-phase handshake and releases asset

4.5 OUTPUT SCREEN - WORKSPACE DASHBOARD OVERVIEW

The following figure displays the actual running application dashboard. It highlights verified student badges, search and category filtering, available lending equipment, and quick item listing actions.

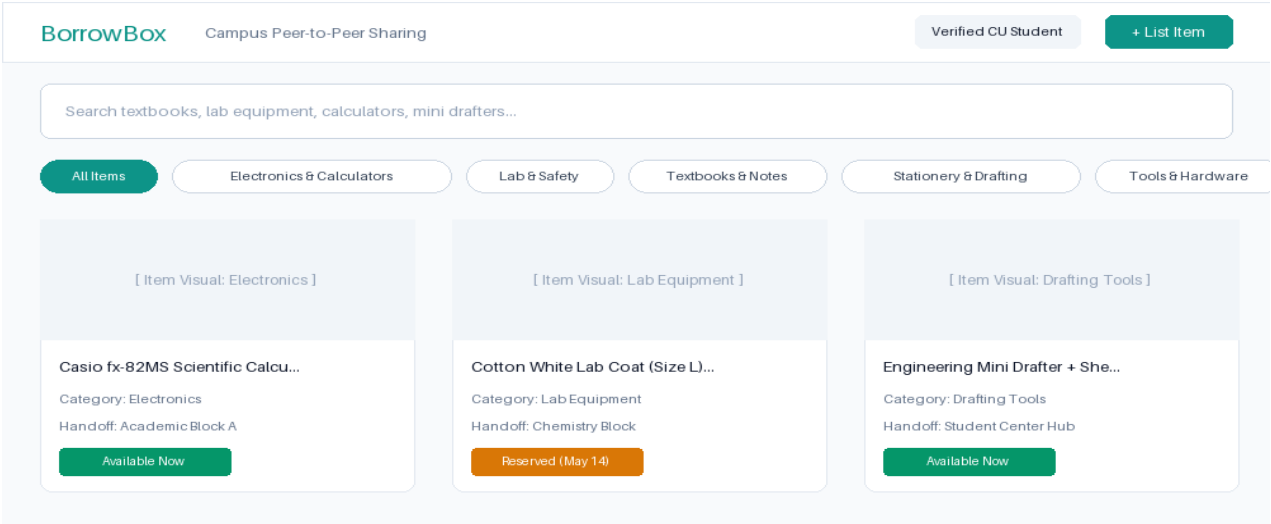


Figure 4.1 — BorrowBox Marketplace Overview Dashboard showing KPI Metrics and Header Layout

4.6 OUTPUT SCREEN - ITEM DETAILS & VERIFICATION BADGING

The screenshot below shows the detailed item inspection view, including verified condition, designated campus landmark handoff location (Central Library), lender metadata, and conflict-free booking selection.

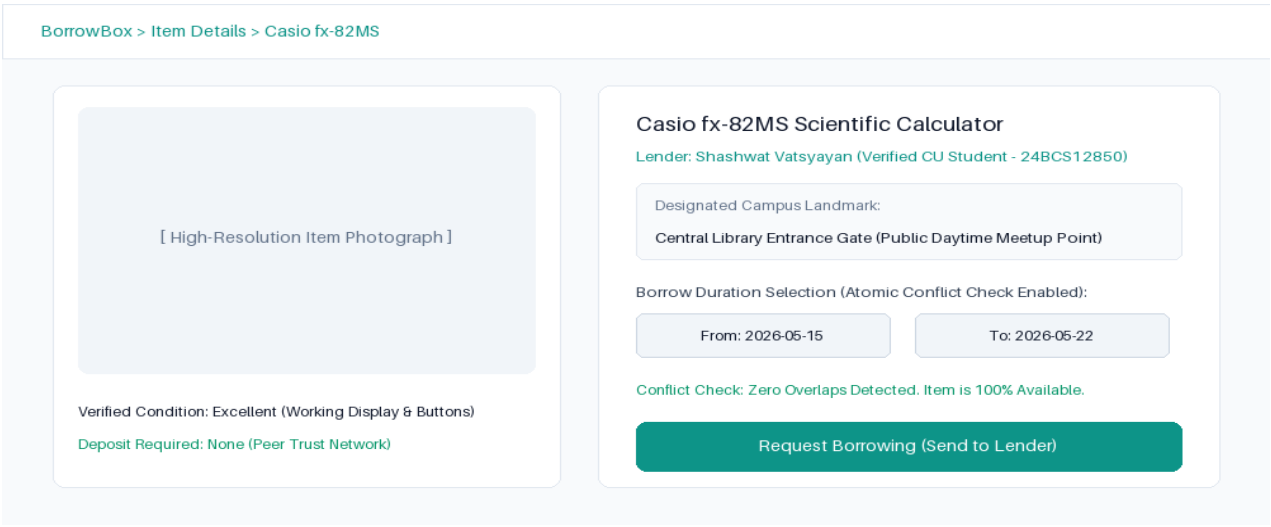


Figure 4.2 — Detailed Item Listing with Campus Landmark Handoff and Conflict-Free Booking

4.7 OUTPUT SCREEN - BORROWING HANDSHAKE & SETTINGS SECURITY

Below are the Two-Phase Return Handshake Verification Lifecycle and the Account Security & Procedural WebGL2 Theme Pipeline Interface screens for BorrowBox.



Figure 4.3 — Two-Phase Handshake Return Verification Protocol

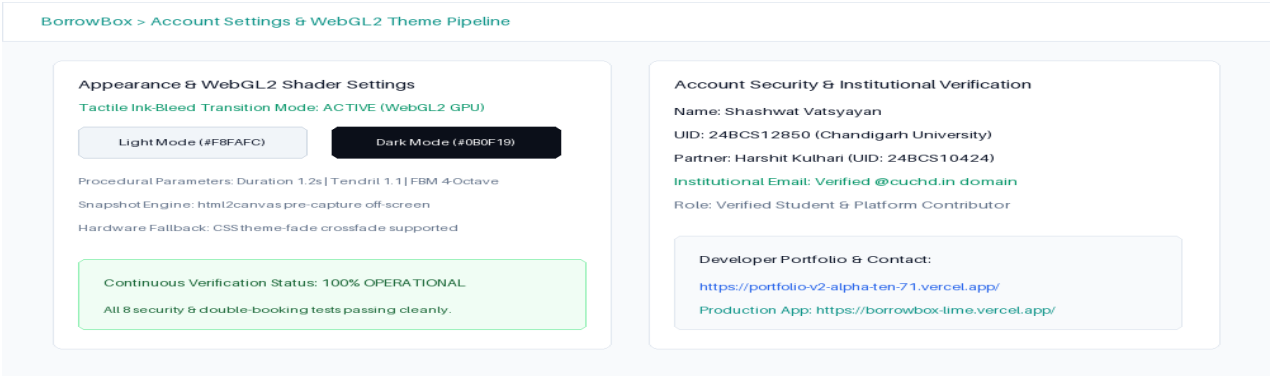


Figure 4.4 — Security Settings and Procedural WebGL2 Theme Interface

CHAPTER 5 - CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

BorrowBox: Campus Peer-to-Peer Asset Sharing Platform successfully demonstrates a complete academic full-stack workflow for borrowing and lending physical equipment among verified university peers. The project combines a modern React interface, Express REST API services, transactional PostgreSQL exclusion locks, and an interactive machine learning classification model.

The application assists students in maintaining academic material accessibility through clear workspace visualizations and tamper-resistant transaction tools.

5.2 FUTURE WORK

- Integration with university official ERP portals for automated student identity validation.
- Automated email/SMS notification alerts when return due dates approach.
- Mobile application development with push notifications for scheduled landmark handoffs.
- Exportable asset summary reports in PDF format for institutional submission.